which is based on the keys/indexes. This isn't how MyISAM works. Data isn't stored in a sorted order. Instead, data is stored in the order in which it is inserted. It is the indexes that are sorted, and these are not stored with data—MyISAM stores data and indexes separately. However, indexes have enough information to point at the exact location of the data in the .MYD file. We will discuss clustered indexes in great detail when talking about the InnoDB storage engine.

Please note that in case a MyISAM table contains multiple indexes, all the indexes are stored in the same .MYI file.

Please bear in mind that even though B-tree provides a facility for very fast lookup of indexes (and thus the data), everything comes at a cost. You must have heard that indexes improve the speed of searches (SELECT family of queries); however, insertions and deletions are slowed down. This is because whenever data is inserted/deleted, MySQL has to alter the indexes in the .MYI file. In case there are no indexes, then there is almost no need to deal with the .MYI part of a table. In this case, one can experience the fastest insertions, however, at the cost of extremely bad performance of SELECT and UPDATE queries. Practically, no application relies on this approach. Indexes must be well-thought-out to avoid the cost of B-tree-driven index management while doing insertions/deletions, and yet getting the best results in case of searches. Whenever dealing with indexes, keep *index selectivity* in mind.

Index selectivity

Though not talked of very often, *index selectivity* is a very critical issue, and when attended properly, can answer two of the most common questions: why isn't MySQL using my index, and why isn't MySQL performing well even when I have properly indexed my table.

Index selectivity, in the simplest words, describes how different the values of a column are. This is assuming that the index contains only one column. In case there are multiple columns in an index, selectivity describes how different the values of those columns are.

Selectivity is a number from 0 to 1—or think of it as a percentage. A value of 1, or 100 per cent, means that each value in the column is unique. This happens with UNIQUE and PRIMARY keys, although non-unique fields may have a selectivity of 1. This depends on the nature of values in the column(s) in question.

SELECTIVITY = NUMBER OF DISTINCT RECORDS / TOTAL NUMBER OF RECORDS

This implies that UNIQUE and PRIMARY indexes always have a selectivity of 1.

The higher the selectivity, the faster the searches are going to be. For example, consider the following table:

Column name	Data type	Indexed?
email_address	VARCHAR (100)	Yes, Primary
password	VARCHAR (100)	No
country_code	INT	Yes

We have two indexes in this table—*email_address* and *country_code*—and 10,000 records. Now let's try to figure out the selectivity:

email_address: Being the primary key, all values in this column must be unique. Hence selectivity is 1, an extremely good index.

country_code: Although there are many countries in this world, I am assuming that you have users coming from 100 countries. Selectivity = 100/10000 = 0.01 or 1 per cent. This is too low to serve as a good index.

Lower selectivity is treated by MySQL as an expensive operation. Higher selectivity is treated as an inexpensive operation.

Please note that the correct/scientific way to calculate selectivity is to use a production-class database, and find the distinct rows using a *COUNT (DISTINCT <COLUMN>)*-like query. If you assume that the number of distinct values in a column do not always work well, avoid heuristics as much as possible.

MySQL has a cost-based optimiser. This means that MySQL calculates the costs of different ways of performing a query, and then chooses the cheapest one. In order to calculate the exact cost, the optimiser would actually have to run the query. If MySQL finds the selectivity of an index (using the above formula) for each query, this will, by all means, kill the machine. So an estimate is taken. This cost-based optimiser uses selectivity when it decides whether or not to use an index. MySQL may not use your index if the selectivity is too low! This means that even when you have an index, MySQL may perform a full-table scan!

An index like *country_code*, which we discussed earlier, will never be used by MySQL. Another major problem is that it will result in wasteful consumption of CPU time during insertions and deletions too. So, pay attention when you create indexes. Every *INSERT* operation will cause multiple B-tree operations to adjust the country code to keep the B-tree balanced; yet, there's no use for this while searching the data.

In the next article in this series, we will discuss the details of the InnoDB storage engine, and how clustered and non-clustered indexes work. END

References

[1] 'High Performance MySQL', Second Edition, O'REILLY, ISBN: 978-0-596-10171-8
[2] MySQL Reference Manual for version 5.1

By: Ankur Kashyap

The author is the chief software architect at G-Cube Solutions Pvt Ltd (www.gc-solutions.net). You can send your queries directly to him at mailtoankur.kashyap@gmail.com.